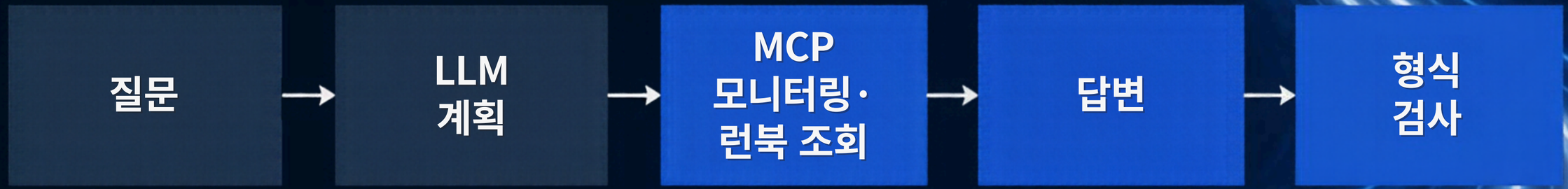
기존 스켈레톤은 어떻게 동작했을까

운영 장애 대응 에이전트의 **기존 처리 흐름**과 검증 방식



기존 구성은 재사용

graph.py 발췌: 답변 길이·번호 형식 검사

```
ok = len(draft) >= 30 and "1)" in draft
```

G028 최민석

스켈레톤에 추가한 처리 흐름

질문부터 설명까지, 기존 연결 위에 **핵심 처리**를 더했습니다.



기존 연동 MCP · 모델 · 웹 연결

추가한 처리 관측 기록 · 정책 판정 · 근거 검증

개선이 필요했던 세 가지

기존 원본과 실행 기록에서 발견한 문제입니다.

01

현재 정상과 안정적인 복구를 구분하지 못함

정상 수치 한 번으로 복구를 확정하는 문제가 있었습니다.

02

이후 요청에서도 조사 대상이 혼동됨

이전 조사 대상을 유지하지 않아 혼선이 발생했습니다.

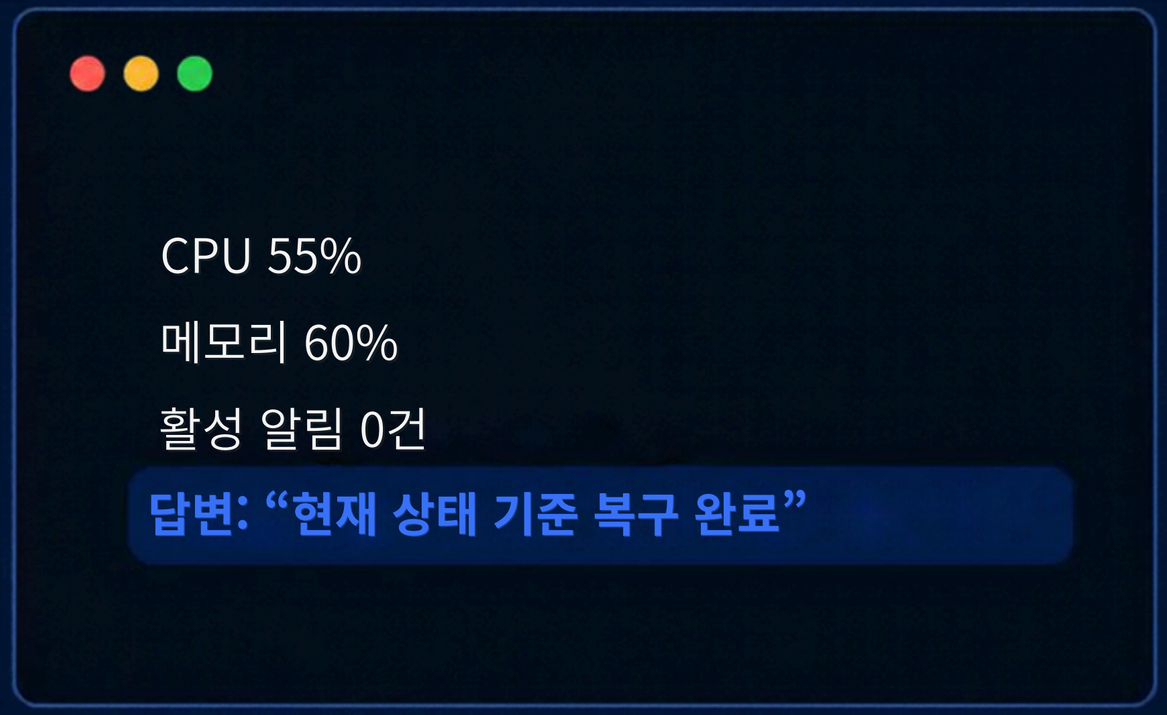
03

실제로 조회하지 않은 자료를 확인한 듯 답변

확인한 근거가 아닌 내용으로 답변한 사례가 있었습니다.

개선 전에는 이렇게 답했습니다

합성 관측과 실제 답변 발췌: 장애 후 정상 첫 관측



당시 판단

현재 상태 기준 복구 완료

하지만

정상 관측은 아직 1회

현재 수치만으로 복구를 확정

이전 상태와 연속 관측 근거 부족

개선 코드는 판단과 설명을 나눴습니다

실제 구현의 핵심 발췌입니다. 판정·근거 검사·재작성 제한을 나눴습니다.

recovery.py

```
normal_streak += 1
```

유효한 정상 관측을 셉니다.
정상 2회·간격 조건으로
복구 여부를 판단합니다.

graph.py

```
draft, errors = validate_draft(
    state["draft"], bundle)
```

grounding.py의 검증 함수로
근거·런북과 모델 선택을
대조합니다.

graph.py · 재작성 조건

```
attempts < 2
```

실패 시 재작성은 최대 1회.
다시 실패하면
보류로 안내합니다.

G028 최민석

같은 관측으로 비교했습니다

같은 질문, **5단계 관측**을 **3회** 반복한 10월 20일 실측 결과입니다.

개선 전 조기 복구 오류 (적을수록 좋음)	3/3	→	개선 후 조기 복구 오류 (적을수록 좋음) 0/3
개선 전 재발 안내 (많을수록 좋음)	0/3	→	개선 후 재발 안내 (많을수록 좋음) 3/3

설명 보완 후 전체 재검증과 실제 API 확인은 별도 검사로 완료했습니다.

최신 로컬 웹 복구	실제 제공 API 진단	일반 장애 정확도를
15/15단계 일치	POST 3건 · SSE 1건 검증 통과	측정한 결과는 아님

G028 최민석

추가 기능으로 조사 기록을 이어갑니다

관측을 이어가며 변화와 재발을 확인하고, 보고서로 남깁니다.

01
새로 진단
대상을 선택하고
새 관측을 저장

→

02
복구 재점검
새 관측으로
회복 상태 재확인

→

03
이력·변화 확인
이전과 현재의 수치와
알림 변화를 비교

→

04
재발 안내
복구 확인 후 이상을
다시 관측하면 재발 안내

→

05
Markdown 보고서
조사 결과를 Markdown 형식으로
저장

예시 **“그 서비스 다시 확인해줘”**라고 말하면, 이전 대상으로 자동 연결됩니다.

G028 최민석

이번 구현의 한계와 다음 단계

샘플 데이터라 실제 운영 복구는 미확인

실제 지표·서비스별 기준 연결은 후속 과제

정상 2회·5초 간격으로 신중해지는 대신 확인 지연

오탐 방지에 유리하지만 복구 판단 도달이 늦어질 수 있음

코드 정책으로 일관성은 확보, 진단 범위는 제한

정책 기반 분류의 장점과 탐지 유연성 사이의 트레이드오프

LangSmith 실제 추적은 미검증

관련 서비스·배포 이력 조사는 미구현 후보

코드 저장소 https://github.com/likefool1714/ai-miniproject